

```
week8_exercise.R x
Source on Save
Run
Source

1 library(readr)
2
3 # Use read_csv() from the readr package to load the data.
4 # It automatically reads the data as a 'tibble' (a modern, user-friendly data frame).
5 student_data <- read_csv("C:/Users/jm166/Desktop/数据科学/week8_exercise/student_data.csv")
6
7 # -----
8 # 1. Show the details of students whose final_exam_mark is greater than 80%.
9 # -----
10 # Use subset() to filter the dataframe based on the condition.
11 high_scorers <- subset(student_data, final_exam_mark > 80)
12
13 print("--- 1. Students with final exam mark > 80% ---")
14 print(high_scorers)
15
17:1 (Untitled) R Script

R 4.6.0 · ~/
>
>
> library(readr)
> # Use read_csv() from the readr package to load the data.
> # It automatically reads the data as a 'tibble' (a modern, user-friendly data frame).
> student_data <- read_csv("C:/Users/jm166/Desktop/数据科学/week8_exercise/student_data.csv")
Rows: 50 Columns: 5
— Column specification —
Delimiter: ","
chr (3): student_id, student_name, gender
dbl (2): coursework_mark, final_exam_mark

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
> # -----
> # 1. Show the details of students whose final_exam_mark is greater than 80%.
> # -----
> # Use subset() to filter the dataframe based on the condition.
> high_scorers <- subset(student_data, final_exam_mark > 80)
> print("--- 1. Students with final exam mark > 80% ---")
[1] "--- 1. Students with final exam mark > 80% ---"
> print(high_scorers)
# A tibble: 3 x 5
  student_id student_name gender coursework_mark final_exam_mark
  <chr>      <chr>      <chr>      <dbl>      <dbl>
1 S007      David Jones Male          64          84
2 S032      Joseph Smith Male          94          82
3 S044      Jessica Brown Female        77          84
> |
```

```
week8_exercise.R x
Source on Save
Run
Source

15
16
17 # -----
18 # 2. Arrange the dataset in ascending order based on course_work_mark.
19 # -----
20 # Use order() to get the sorted indices, then apply them to the dataframe rows.
21 sorted_dataset <- student_data[order(student_data$coursework_mark), ]
22
23 print("--- 2. Dataset sorted by course_work_mark (Ascending) ---")
24 print(sorted_dataset)
25
26
27:1 # (Untitled) R Script

>
>
>
>
>
>
>
>
> # -----
> # 2. Arrange the dataset in ascending order based on course_work_mark.
> # -----
> # Use order() to get the sorted indices, then apply them to the dataframe rows.
> sorted_dataset <- student_data[order(student_data$coursework_mark), ]
> print("--- 2. Dataset sorted by course_work_mark (Ascending) ---")
[1] "--- 2. Dataset sorted by course_work_mark (Ascending) ---"
> print(sorted_dataset)
# A tibble: 50 x 5
  student_id student_name   gender coursework_mark final_exam_mark
  <chr>      <chr>         <chr>         <dbl>         <dbl>
1 S028      Charles Garcia   Male           34            39
2 S001      Elizabeth Miller Female          40            41
3 S006      Sarah Rodriguez  Female          41            35
4 S036      Karen Jones      Female          41            45
5 S038      Elizabeth Miller Female          41            34
6 S037      David Davis      Male           42            38
7 S030      Barbara Williams Female          43            55
8 S033      Richard Miller   Male           43            42
9 S040      Sarah Miller     Female          43            51
10 S043      Patricia Williams Female          46            38
# i 40 more rows
# i Use `print(n = ...)` to see more rows
>
>
> |
```

The screenshot shows the RStudio environment. The top pane displays an R script with the following code:

```

27 # -----
28 # 3. Display only the student name and final_exam_mark attributes.
29 # -----
30 # Extract specific columns by passing a vector of column names.
31 selected_attributes <- student_data[, c("student_name", "final_exam_mark")]
32
33 print("--- 3. Specific attributes (Name and Final Exam Mark) ---")
34 print(selected_attributes)
35
36
37:1 # (Untitled)

```

The bottom pane shows the console output of the script:

```

>
>
> # -----
> # 3. Display only the student name and final_exam_mark attributes.
> # -----
> # Extract specific columns by passing a vector of column names.
> selected_attributes <- student_data[, c("student_name", "final_exam_mark")]
> print("--- 3. Specific attributes (Name and Final Exam Mark) ---")
[1] "--- 3. Specific attributes (Name and Final Exam Mark) ---"
> print(selected_attributes)
# A tibble: 50 x 2
  student_name    final_exam_mark
  <chr>          <dbl>
1 Elizabeth Miller      41
2 William Garcia       63
3 Sarah Johnson        64
4 William Davis        62
5 John Smith           56
6 Sarah Rodriguez      35
7 David Jones          84
8 Thomas Martinez      60
9 Michael Garcia       64
10 Jessica Garcia       65
# i 40 more rows
# i Use `print(n = ...)` to see more rows
>
>
>
>
>
>
>
>
>
>

```

```
week8_exercise.R
Source on Save
Run
Source

32
33 print("--- 3. Specific attributes (Name and Final Exam Mark) ---")
34 print(selected_attributes)
35
36
37 # -----
38 # 4. Specify the number of rows and columns in the dataset.
39 # -----
40 # nrow() returns the number of rows, ncol() returns the number of columns.
41 num_rows <- nrow(student_data)
42 num_cols <- ncol(student_data)
43
44 cat("--- 4. Dimensions of the dataset ---\n")
45 cat("Number of Rows:", num_rows, "\n")
46 cat("Number of Columns:", num_cols, "\n\n")
47
48
49:1 (Untitled) R Script

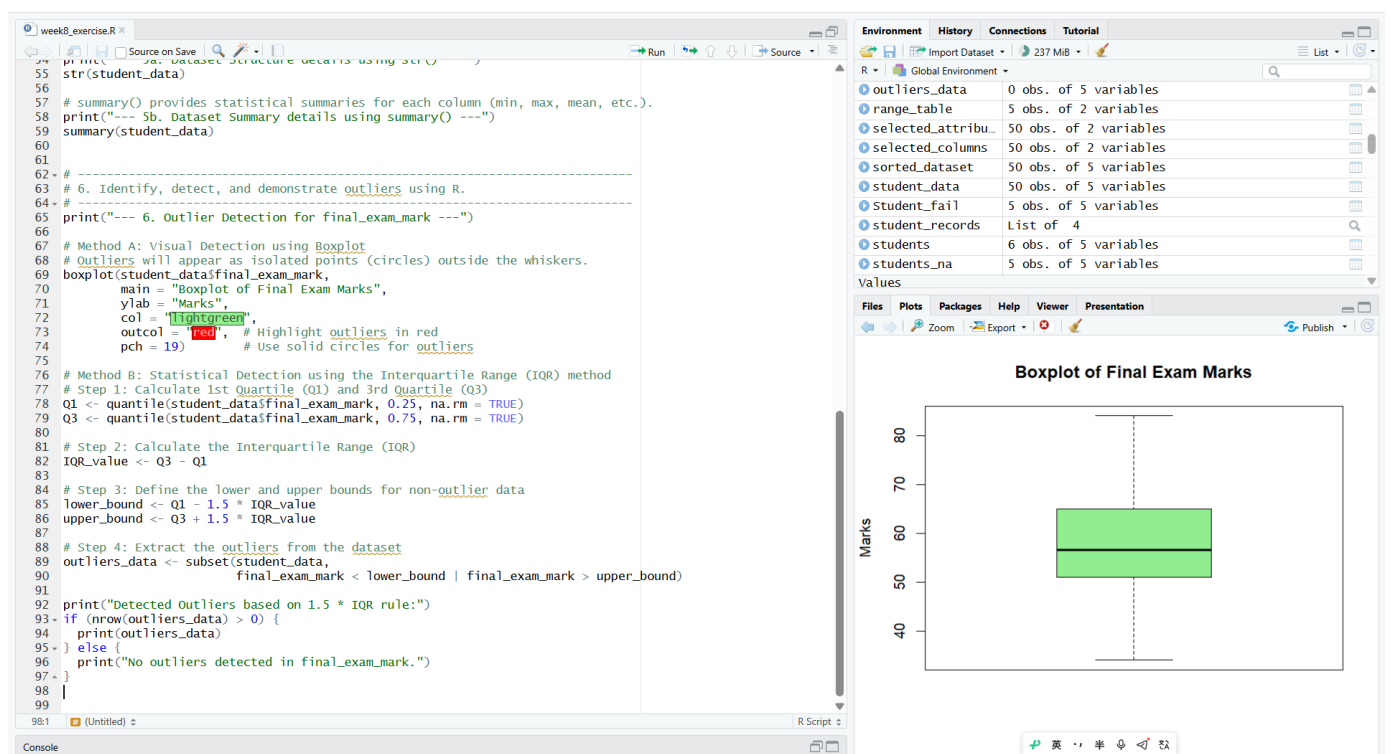
R 4.6.0
8 Thomas Martinez 60
9 Michael Garcia 64
10 Jessica Garcia 65
# i 40 more rows
# i Use `print(n = ...)` to see more rows
>
>
>
>
>
>
>
>
> # -----
> # 4. Specify the number of rows and columns in the dataset.
> # -----
> # nrow() returns the number of rows, ncol() returns the number of columns.
> num_rows <- nrow(student_data)
> num_cols <- ncol(student_data)
> cat("--- 4. Dimensions of the dataset ---\n")
--- 4. Dimensions of the dataset ---
> cat("Number of Rows:", num_rows, "\n")
Number of Rows: 50
> cat("Number of Columns:", num_cols, "\n\n")
Number of Columns: 5
> |
```

```
week8_exercise.R x
Source on Save
Run
Source

49 #
50 # 5. Explain how to obtain details and show dataset details.
51 # -----
52 # str() shows the structure of the dataset (data types, preview of values).
53 # Since it's a tibble, you could also just use glimpse(student_data) if you load dplyr.
54 print("--- 5a. Dataset Structure details using str() ---")
55 str(student_data)
56
57 # summary() provides statistical summaries for each column (min, max, mean, etc.).
58 print("--- 5b. Dataset Summary details using summary() ---")
59 summary(student_data)
60

R 4.6.0 · ~/
> # 5. Explain how to obtain details and show dataset details.
> # -----
> # str() shows the structure of the dataset (data types, preview of values).
> # Since it's a tibble, you could also just use glimpse(student_data) if you load dplyr.
> print("--- 5a. Dataset Structure details using str() ---")
[1] "--- 5a. Dataset Structure details using str() ---"
> str(student_data)
spec_tbl_ [50 x 5] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ student_id   : chr [1:50] "S001" "S002" "S003" "S004" ...
 $ student_name : chr [1:50] "Elizabeth Miller" "William Garcia" "Sarah Johnson" "William Davis" ...
 $ gender       : chr [1:50] "Female" "Male" "Female" "Male" ...
 $ coursework_mark: num [1:50] 40 59 79 56 74 41 64 73 70 67 ...
 $ final_exam_mark: num [1:50] 41 63 64 62 56 35 84 60 64 65 ...
- attr(*, "spec")=
.. cols(
..   student_id = col_character(),
..   student_name = col_character(),
..   gender = col_character(),
..   coursework_mark = col_double(),
..   final_exam_mark = col_double()
.. )
- attr(*, "problems")=<pointer: 0x000001e992cb89b0>
> # summary() provides statistical summaries for each column (min, max, mean, etc.).
> print("--- 5b. Dataset Summary details using summary() ---")
[1] "--- 5b. Dataset Summary details using summary() ---"
> summary(student_data)
  student_id student_name gender coursework_mark final_exam_mark
Length :50  Length :50  Length :50  Min. :34.00  Min. :34.00
N.unique :50  N.unique :42  N.unique : 2  1st Qu.:51.00  1st Qu.:51.50
N.blank : 0  N.blank : 0  N.blank : 0  Median :62.00  Median :56.50
Min.nchar: 4  Min.nchar:10  Min.nchar: 4  Mean :62.42  Mean :57.88
Max.nchar: 4  Max.nchar:17  Max.nchar: 6  3rd Qu.:73.00  3rd Qu.:64.75
Max. :97.00  Max. :84.00

> |
```



```

R 4.6.0 · ~/
>
>
>
>
>
>
> # -----
> # 6. Identify, detect, and demonstrate outliers using R.
> # -----
> print("--- 6. Outlier Detection for final_exam_mark ---")
[1] "--- 6. Outlier Detection for final_exam_mark ---"
> # Method A: Visual Detection using Boxplot
> # Outliers will appear as isolated points (circles) outside the whiskers.
> boxplot(student_data$final_exam_mark,
+         main = "Boxplot of Final Exam Marks",
+         ylab = "Marks",
+         col = "lightgreen",
+         outcol = "red", # Highlight outliers in red
+         pch = 19)      # Use solid circles for outliers
> # Method B: Statistical Detection using the Interquartile Range (IQR) method
> # Step 1: Calculate 1st Quartile (Q1) and 3rd Quartile (Q3)
> Q1 <- quantile(student_data$final_exam_mark, 0.25, na.rm = TRUE)
> Q3 <- quantile(student_data$final_exam_mark, 0.75, na.rm = TRUE)
> # Step 2: Calculate the Interquartile Range (IQR)
> IQR_value <- Q3 - Q1
> # Step 3: Define the lower and upper bounds for non-outlier data
> lower_bound <- Q1 - 1.5 * IQR_value
> upper_bound <- Q3 + 1.5 * IQR_value
> # Step 4: Extract the outliers from the dataset
> outliers_data <- subset(student_data,
+                          final_exam_mark < lower_bound | final_exam_mark > upper_bound)
> print("Detected Outliers based on 1.5 * IQR rule:")
[1] "Detected Outliers based on 1.5 * IQR rule:"
> if (nrow(outliers_data) > 0) {
+   print(outliers_data)
+ } else {
+   print("No outliers detected in final_exam_mark.")
+ }
[1] "No outliers detected in final_exam_mark."
>
>
> |

```